Xi'an Jiaotong-Liverpool University

西交利物浦大学

PAPER CODE	EXAMINER	DEPARTMENT	TEL
CPT111	DR. ERICK PURWANTO and DR. TENG MA	Computing	81889179

FIRST SEMESTER AY2324 FINAL EXAMINATION

BACHELOR DEGREE – Year 2

INTRODUCTION TO JAVA

TIME ALLOWED: 2 Hours

INSTRUCTIONS TO CANDIDATES

1、 **Total marks available are 100.**
This will count for 40% of the final module mark.

2、 **Answer all questions.**

3、 **All questions should be answered on the Learning Mall Quiz Page.**

4、 **All the answers must be in English.**

<u>Part 1 Multiple-Choice Questions:</u> 50 marks, 25 Questions

1. Which statement about **method overloading** is correct?
   A. It requires the same return type only.
   B. It happens in subclasses with identical signatures.
   C. It uses the same name with different parameter lists in the **same** class.
   D. It requires the @Override annotation.

2. What does this print?

```
String s = "";
System.out.println(s.length());
```

   A. nothing
   B. 0
   C. 1
   D. 2

3. Which expression correctly implements boolean **XOR** for boolean a, b?
   A. (a && b) || (!a && !b)
   B. (!a && b) || (a && !b)
   C. (a || b) && (a && b)
   D. !(a || b)

4. Given String t = "Halloween";, what is the value of t.substring(2,5)?
   A. "all"
   B. "llo"
   C. "owe"
   D. Throws an exception

5. Which call finds the **last** occurrence of "e" in "Halloween"?
   A. cosmo.indexOf("e")
   B. cosmo.lastIndexOf("e")
   C. cosmo.indexOf("e", cosmo.length()-1)
   D. cosmo.lastIndexOf("e", 0)

6. Which is **true** about equalsIgnoreCase?
   A. It compares by reference.
   B. It compares strings while ignoring case differences.
   C. It only works for ASCII letters.
   D. It throws on null.

7. What exception is appropriate to throw if a method receives a **zero divisor** in integer division?
   A. NumberFormatException
   B. IllegalArgumentException
   C. ArithmeticException
   D. ArrayIndexOutOfBoundsException

8. In the Swordsman/Pillar example, which call from Pillar.attackDamage() invokes the **overridden** superclass method?
   A. attackDamage()
   B. this.attackDamage()
   C. super.attackDamage()
   D. Swordsman.attackDamage()

9. If a class variable should be shared by **all** instances, how should it be declared?
   A. private final
   B. public
   C. static (possibly with a visibility modifier)
   D. transient

10. What does **casting** do in int y = (int) x; where x is a double?
    A. Rounds to the nearest int
    B. Truncates the fractional part
    C. Throws unless x is an integer
    D. Performs banker's rounding

11. Which statement about **constants** is correct?
    A. Declare with final only.
    B. Use static final so one value is shared across instances.
    C. Java has no true constants.
    D. Constants must be public.

12. What does this print?

```
double r = Math.random() * 6;
int n = (int) r;
System.out.println(n);
```

    A. 1..6 inclusive
    B. 0..6 inclusive
    C. 0..5 inclusive
    D. 1..5 inclusive

13. Which split usage correctly splits by spaces after trimming?
    A. s.split("")
    B. s.trim().split(" ")
    C. s.trim().split("\\s+") (after trimming or not)
    D. s.replace(" ", "")

14. Which line safely avoids StringIndexOutOfBoundsException before taking str.substring(0, 5)?
    A. if (str.length() > 5)
    B. if (str.length() >= 5)
    C. if (str.length() == 5)
    D. // no check needed

15. In Java access modifiers, which order goes from **least** to **most** visible?
    A. default → private → protected → public
    B. private → default → protected → public
    C. public → protected → default → private
    D. private → protected → default → public

16. Why is @Override useful on overriding methods?
    A. It improves runtime performance.
    B. It enforces access control.
    C. It triggers a compile-time error if the signature doesn't actually override.
    D. It allows overloading with the same signature.

17. Consider:

```
public static int addTwoNumbers(int a, int b) {
int result = a + b;
return result;
}
```

Which is **true**?
A. It returns void.
B. It has two parameters and returns an int.
C. It must be called from a constructor.
D. It cannot be called twice.

18. What is printed?

```
int[] arr = new int[4];
arr[0] = 2; arr[1] = 3; arr[2] = 4; arr[3] = 5;
System.out.println(arr.length);
```

A. 3
B. 4
C. 5
D. Compile error

19. What happens if you execute str.charAt(str.length())?
    A. Returns '\0'
    B. Returns last character
    C. Throws a string index out of bounds exception
    D. Wraps to index 0

20. Which constructor correctly uses this to chain and set a field?
    A.

```
public Swordsman(String n){ this.name = n; }
public Swordsman(){ Swordsman("Nameless"); }
```

    B.

```
public Swordsman(){ this("Nameless"); }
public Swordsman(String n){ name = n; }
```

    C.

```
public Swordsman(String n){ name = n; }
public Swordsman(){ super(n); }
```

    D.

```
public Swordsman(){ name = "Nameless"; this(); }
```

21. Which line best initializes **HP = 100** inside a constructor that tracks health?
    A. double healthPoint == 100.0;
    B. healthPoint = 100.0;
    C. private double healthPoint = 100.0;
    D. public double healthPoint = 100.0;

22. Which statement about **protected** is correct?
    A. Accessible only within the same class.
    B. Accessible in same package and in subclasses (even in different packages).
    C. Accessible everywhere.
    D. Not allowed on methods.

23. To **catch** an IllegalArgumentException from receiveDamage, which header fits?
    A. catch (IllegalArgumentException e)
    B. catch (IllegalArgumentException())
    C. catch (Exception())
    D. catch (IllegalArgumentException iae) { throw; }

24. What's the correct way to convert "123" to an int?
    A. (int) "123"
    B. Double.parseDouble("123")
    C. Integer.parseInt("123")
    D. Integer.toString("123")

25. In the following code, what will be printed?

```
int result = 10;
addTwoNumbers(2,5);
System.out.println(result);
```

Assume addTwoNumbers returns num1 + num2 but the return value isn't stored.

A. 2
B. 5
C. 7
D. 10

<u>Part 2 Code Description and Completion</u>: 50 marks, 4 Questions.

Question 1 (10 marks)

Look at the code below, read it and then complete the code description below by filling in the spaces.

```
public class LoopExample {
   public static void main(String[] args) {
      int sum = calculateSum(5);
      System.out.println("The sum of numbers from 1 to 5 is: " + sum);
   }

   public static int calculateSum(int n) {
      int sum = 0;
      for (int i = 1; i <= n; i++) {
         sum += i;
      }
      return sum;
   }
}
```

This is a Java code snippet uses a for loop within the method _____ to calculate the sum of several numbers. The method takes an integer parameter named _____ and initializes a variable _____ to store the result. It then iterates through a for loop that starts from _____, continues as long as _____ is less than or equal to _____, and incremented by _____ in each iteration. Inside the loop, it adds the value of _____ to _____. The method returns the final result of sum equals to _____.

Question 2 (20 marks)

a) Create a class to store data about a Module. A Module has a moduleName, a moduleCode, and a requirement on creditHours, and all these three variables should not be modified by unauthorized personnel. You should use sensible variable types and names, and create instance variables, a constructor, and an appropriate encapsulation method to show a formatted string containing the module's information. (10 marks)

```
public class _______ {
   private String moduleName;
   private String moduleCode;
   _______ int _______;

   public _______ (String moduleName, _______ moduleCode, _______ creditHours) {
      this.moduleName = _______;
      _______.moduleCode = moduleCode;
      this.creditHours = creditHours;
   }

   public _______ getModuleInfo() {
      return "Module Name: " + moduleName + ", Code: " + moduleCode + ", Credit
Hours: " + _______;
   }

}
```

b) Declare the method named moduleRank that ranks the difficulty of module based on its credit hours. If the module has more than 4 credit hours, it's considered advanced; else if it has more than 2 credit hours, it's considered medium, otherwise, it's considered low. (4 marks)

```
public _______ moduleRank() {
   if (creditHours > _______){
      return "Advanced";
   } else if (_______ > 2) {
      return "Medium";
   } else {
      return _______;
   }
}
```

c) Now we are creating a new class named **Course** to represent a course with a fixed number of modules. Now, you are asked to implement this class to store the modules in an array and the instance of Course should be able to add individual modules into the array one by one. (6 marks)

```
public class Course {
   private String courseName;
   private _______ [] modules;
   private int addedModuleCount;

   public Course(String courseName, int numOfModules) {
      this.courseName = courseName;
      modules = _______ Module[_______];
      this.addedModuleCount = 0;
   }

   public void addModule(_______ module) {
      if (addedModuleCount < _______ . _______){
         this.modules[_______] = module;
      }
   }

}
```

Question 3: (10 marks)

Complete the following Java function method that given a string contained *only letters*, returns true if the string is composed of all uppercase letters, and false otherwise, **using recursion**.

On input empty string, the function also returns true.
Do **not** use any loops. Submissions with loops will **not** pass.

<u>Sample test cases:</u>
String str1 = "ABCDE";
System.out.println(isAllUppercase(str1)); → true
String str2 = "XYz";
System.out.println(isAllUppercase(str2)); → false

```
public static boolean isAllUppercase(String str) {
      // base case


      // recursive step


   }
```

Question 4: (10 marks)

Complete the following class Vehicle and its subclass Car that leverage **inheritance** and **overriding** in their implementation. (10 marks)

```
public class Vehicle {
   _______ String brand;
   _______ int year;

   public _______ (String brand, int year) {
      this.brand = brand;
      this.year = year;
   }

   public _______ _______ () {
      System.out.println("Brand: " + brand + ", Year: " + year);
   }
}

public class Car extends Vehicle {
   _______ String model;

   _______ Car(String brand, int year, String model) {
         _______ (_______, _______); // Call the constructor of the superclass
         _______.model = model;
      }

      @_______
      public void displayInfo() {
         System.out.println("Brand: " + _______ _______ _______ + ", Year: " + _______ _______ _______ +
", Model: " + model);
      }
   }
}

public class FinalExam {
   public static void main(String[] args) {
      Vehicle myCar = new Car("Toyota", 2018, "Camry");
      myCar.displayInfo();
```

END OF FINAL EXAM